Distributed Task Execution and Gossip Protocol Simulation in an OMNeT++ CHORD-Inspired Peer-to-Peer Network

CSL3080 Computer Networks (Assignment–2)

Group Members: Namya Dhingra (B23CS1040)
Bhargav Shekokar (B23CS1008)

April 17, 2026

Contents

1 Introduction **2**

2 Problem Statement **2**

3 System Design / Architecture **2**
3.1 High-level components . . . 2
3.2 Module interaction diagram . . . 3

4 Network Topology Explanation **3**
4.1 Topology representation . . . 3
4.2 Ring plus CHORD-inspired shortcut links . . . 3

5 Algorithm / Workflow **4**
5.1 Task splitting and assignment . . . 4
5.2 Routing rule (greedy clockwise distance) . . . 4
5.3 Aggregation and completion . . . 5
5.4 Gossip dissemination and termination . . . 5

6 Optimization Technique: $O(N) \to O(\log N)$ (CHORD-inspired) **5**
6.1 Baseline: ring-only routing . . . 5
6.2 Overlay augmentation using finger links . . . 5
6.3 Greedy routing complexity . . . 5

7 Gossip Protocol Explanation **6**
7.1 Message format . . . 6
7.2 Duplicate suppression (ML set) . . . 6

8 Implementation Details **7**
8.1 Directory structure (key files) . . . 7
8.2 Runtime parameters . . . 7
8.3 Coordinator topology wiring . . . 7
8.4 Client node behavior and state . . . 8

9 Results and Output **8**
9.1 Observed output structure . . . 8
9.2 Example excerpt . . . 9
9.3 Interpretation . . . 10

10 Conclusion **10**

Abstract

This report describes an OMNeT++ discrete-event simulation of a fully distributed peer-to-peer network that (i) executes a simple distributed computation (maximum of an array) by splitting it into multiple subtasks and (ii) spreads task-completion notifications using a loop-free gossip protocol. The overlay is a ring augmented with CHORD-inspired shortcut links ("fingers") loaded from an editable topology file. Task and result messages are forwarded using greedy routing based on clockwise ring distance; with finger links present, routes typically require $O(\log N)$ hops rather than an $O(N)$ ring walk. After a node consolidates all subtask results for its task, it emits a gossip message and each peer forwards it at most once using a local message log to suppress duplicates. The remainder of this report explains the design, topology, algorithms, and the output produced by the simulation.

1 Introduction

This assignment extends a ring-based peer-to-peer gossip setting into a fully peer-to-peer system that also supports remote task execution. A ring topology is easy to maintain and guarantees connectivity, but it is inefficient for targeted delivery: in the worst case, reaching a specific node in a ring takes $\Theta(N)$ hops.

To address this, the network is augmented with additional links inspired by CHORD. These shortcut (finger) links reduce the number of hops required for directed message delivery. On top of the overlay, each node periodically initiates a task, divides it into more subtasks than the number of nodes ($x > N$), routes each subtask to node ($i \bmod N$), and aggregates the returned results. Once a task completes, a loop-free gossip protocol disseminates a completion message across the overlay.

2 Problem Statement

Given N client nodes connected in a ring topology, each node must:

1. Divide a task into x subtasks with IDs $i \in \{0, \ldots, x-1\}$, where $x > N$.
2. Send subtask i to client ($i \bmod N$).
3. For this assignment, execute a simple distributed task: compute the maximum of an array of k elements by splitting the array into x parts (with $k/x \geq 2$).
4. Collect all subtask results and compute the final result.
5. After task completion, generate a gossip message of the form $\langle t : IP : ID \rangle$ and disseminate it via a loop-free gossip protocol.

Optimization requirement. In a pure ring, reaching node ($i \bmod N$) can require $O(N)$ hops. The system must be optimized to $O(\log N)$ message exchanges by adding additional connections inspired by CHORD.

3 System Design / Architecture

3.1 High-level components

The implementation is split into the following components:

- **NED network description:** instantiates N `ClientNode` peers and a single `Coordinator` module.
- **Coordinator (topology builder):** reads the edge list in `topo.txt` and wires up bidirectional links between peer gates at initialization.
- **ClientNode (peer logic):** generates subtasks, forwards messages, computes local maxima, aggregates results, and runs gossip-based completion dissemination.
- **Application message:** a single OMNeT++ message type (`ChordAppMsg`) with a `msgType` selector for task/result/gossip semantics.

3.2 Module interaction diagram

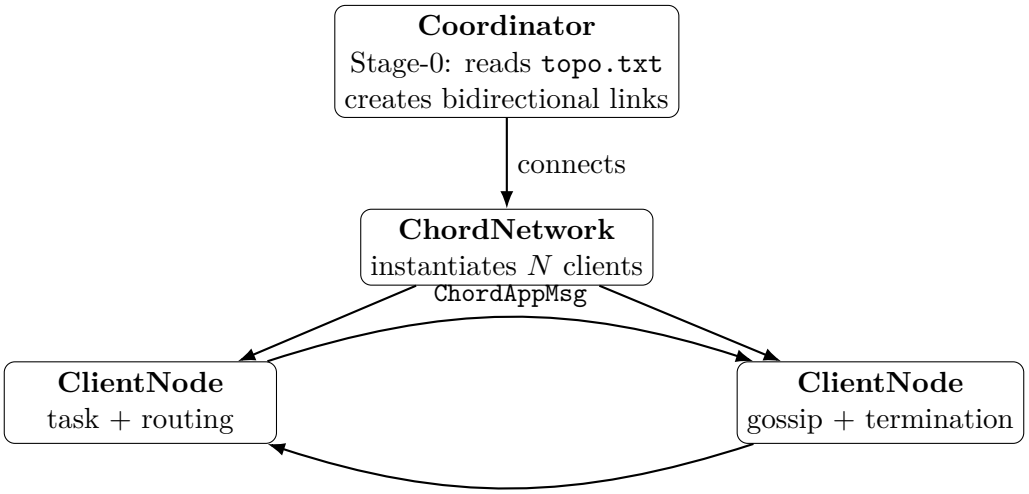


Figure 1: System architecture in OMNeT++: the coordinator constructs the overlay, and clients exchange typed application messages over the created links.

4 Network Topology Explanation

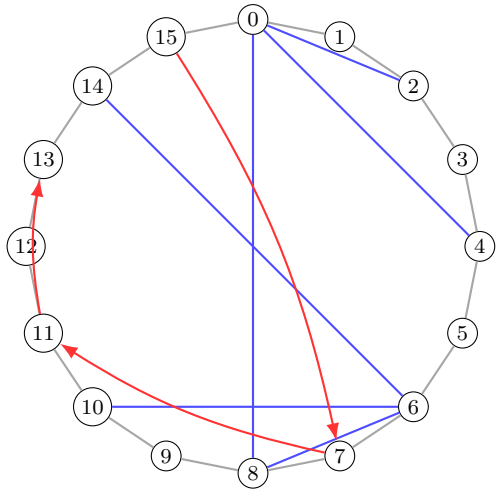
4.1 Topology representation

Connectivity is provided as an undirected edge list in `topo.txt`. Each line specifies a pair (u, v), and the coordinator instantiates two directed links (one in each direction) so that the resulting overlay behaves as an undirected graph.

4.2 Ring plus CHORD-inspired shortcut links

The topology contains:

- **Ring edges:** $(i, (i + 1) \bmod N)$, ensuring connectivity.
- **Finger edges:** $(i, (i + 2^j) \bmod N)$ for $j = 0, 1, \ldots, \lfloor \log_2 N \rfloor$, providing long-range shortcuts.



Legend: gray = ring edges; blue = example finger edges; red = example shortcut route.

Figure 2: Illustration of the overlay for $N = 16$: a connected ring augmented with CHORD-inspired shortcut (finger) links. In the implementation, all edges come from `topo.txt`.

Assumption. The provided topology generator script constructs all ring and finger edges for powers of two; evaluators may edit `topo.txt` to test alternative overlays without recompiling.

5 Algorithm / Workflow

5.1 Task splitting and assignment

Each peer initiates a task after a small randomized start delay (a self-message scheduled at initialization). At runtime it reads k and x from `config.txt`, generates k random integers in $[1, 1000]$, and partitions them into x chunks. For each subtask ID i:

- The target node is $d = i \bmod N$.
- A `TASK_MSG` is sent to destination d, carrying the sub-array chunk.
- The destination computes a local maximum and returns a `RESULT_MSG` to the initiator.

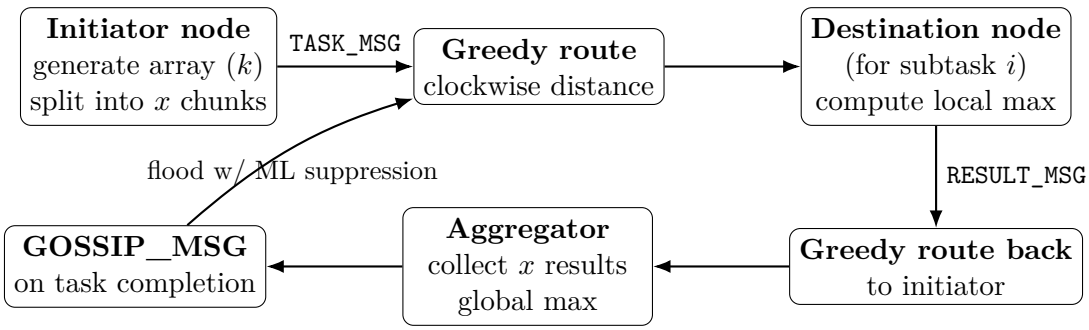


Figure 3: End-to-end workflow: a task is split into subtasks, routed to destinations for local computation, aggregated at the initiator, and followed by completion gossip dissemination.

5.2 Routing rule (greedy clockwise distance)

Nodes do not store an explicit finger table in code. Instead, forwarding decisions are made using the set of currently connected neighbors (i.e., the endpoints of `port[]` gates). The routing

metric is the clockwise ring distance:

$$\text{dist}(a, b) = (b - a + N) \bmod N.$$

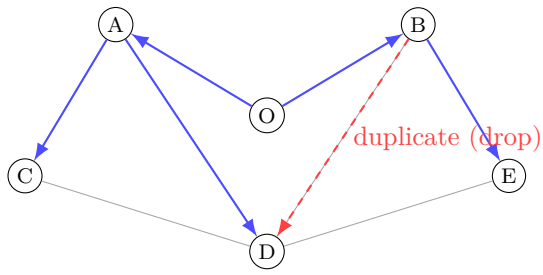
At each hop, the node prefers a neighbor that strictly reduces the remaining distance. If none exists (e.g., due to missing edges or an avoid constraint), it forwards to the neighbor that yields the smallest distance as a fallback.

5.3 Aggregation and completion

The initiator counts returned results until all x results arrive, and tracks the maximum across the received local maxima. Upon completion, it constructs a `GOSSIP_MSG` containing: $\langle$`gossipTimestamp` : `gossipOriginIP` : `gossipClientId`$\rangle$.

5.4 Gossip dissemination and termination

Each node stores a message log (ML) keyed by (`originIP`, `timestamp`). On first receipt, it logs the message and forwards it to all neighbors except the sender. Duplicates are dropped immediately. A node terminates after observing at least N unique completion gossips (one originating from each client).



Legend: blue = first-time forwards; red dashed = duplicate suppressed by ML.

Figure 4: Loop-free gossip in a connected overlay: the same gossip may reach a node through multiple paths, but duplicates are dropped using the local message log (ML).

6 Optimization Technique: $O(N) \to O(\log N)$ (CHORD-inspired)

6.1 Baseline: ring-only routing

In a pure ring overlay, routing a message from a source to a target node may require up to $N - 1$ hops in the worst case (clockwise traversal), which is $\Theta(N)$.

6.2 Overlay augmentation using finger links

The topology augments the ring with shortcut links where node i connects to nodes at offsets 2^j (modulo N). Intuitively, these links allow a message to "jump" across exponentially increasing distances.

6.3 Greedy routing complexity

With fingers present, greedy forwarding reduces the remaining clockwise distance much faster than single-step ring traversal. In a CHORD-like overlay, the remaining distance decreases geometrically, leading to $O(\log N)$ expected hops for delivery. This satisfies the assignment requirement while keeping routing logic local (each node only looks at its immediate neighbors).

Algorithm 1 Greedy routing used by `ClientNode::routeMessage`

Require: current node id u, destination id d, neighbor set $\mathcal{N}(u)$, optional avoid id a

$best \leftarrow$ NONE; $bestDist \leftarrow \mathrm{dist}(u, d)$
$fallback \leftarrow$ NONE; $fallbackDist \leftarrow +\infty$
for all $v \in \mathcal{N}(u)$ **do**
    $distVD \leftarrow \mathrm{dist}(v, d)$
    if $v \neq a$ **and** $distVD < bestDist$ **then**
        $best \leftarrow v$; $bestDist \leftarrow distVD$
    if $distVD < fallbackDist$ **then**
        $fallback \leftarrow v$; $fallbackDist \leftarrow distVD$
if $best =$ NONE **then**
    $best \leftarrow fallback$
Forward message to neighbor $best$

7 Gossip Protocol Explanation

The gossip mechanism is designed to (i) reach every node in a connected overlay and (ii) avoid persistent circulation and unnecessary retransmissions.

7.1 Message format

Each node creates a completion gossip message:

$$M = \langle t : IP : ID \rangle,$$

where t is the origin node's simulation timestamp at creation time. In this implementation, the "IP" field is represented by the origin node ID, and ID is the client identifier.

7.2 Duplicate suppression (ML set)

Each node maintains a set ML of already-seen gossip keys. In this implementation, the key is `(gossipOriginIP, gossipTimestamp)`. On receiving a gossip message:

- If the key is already in ML, discard the message.
- Otherwise, insert into ML, log it, and forward it to all neighbors except the sender.

Algorithm 2 Loop-free gossip forwarding

Require: node u, received gossip M with sender id s

$key \leftarrow (M.originIP, M.timestamp)$
if $key \in ML$ **then**
    return ▷ duplicate
$ML \leftarrow ML \cup \{key\}$
Log M with local timestamp `simTime()`
for all $v \in \mathcal{N}(u)$ **do**
    if $v \neq s$ **then**
        send copy of M to v
if $|ML| \geq N$ **then**
    mark node as terminated

8 Implementation Details

8.1 Directory structure (key files)

Table 1 lists the main files that implement the simulation.

Build and execution steps (including common troubleshooting) are summarized in the project README [4].

File	Purpose
`Network.ned`	Defines the network with N clients and one coordinator.
`ClientNode.ned`	Declares `ClientNode` with variable-size `port[]` gates.
`Coordinator.ned`	Declares the coordinator module.
`ChordAppMsg.msg`	Defines `ChordAppMsg` and message fields for task/result/-gossip.
`Coordinator.cc`	Reads `topo.txt` and creates bidirectional delay channels between nodes.
`ClientNode.cc/.h`	Core logic: task splitting, greedy routing, aggregation, gossip, termination.
`generate_topo.py`	Generates `config.txt` and `topo.txt` with ring + finger edges.
`config.txt`	Stores N, k, and x (runtime parameters read by nodes).
`topo.txt`	Editable overlay edge list.
`outputfile.txt`	Console-style output captured to file.

Table 1: Key files in the implementation.

8.2 Runtime parameters

In the reference configuration used for the provided run:

- $N = 16$ nodes, configured in `omnetpp.ini` and also written to `config.txt`.
- $k = 100$ array elements per task.
- $x = 20$ subtasks per task.

Parameter	Symbol	Value
Number of clients	N	16
Array size	k	100
Subtasks	x	20
Link delay (per hop)	δ	0.01 s

Table 2: Reference simulation parameters (from `config.txt` and coordinator channel delay).

8.3 Coordinator topology wiring

The coordinator executes at initialization stage 0. For each edge (u, v) in `topo.txt`, it finds an unconnected `port` gate on each endpoint and connects the two modules using a delay channel (and then repeats in the reverse direction). This design keeps the overlay fully editable through `topo.txt`: changing the neighbor set does not require regenerating NED files.

8.4 Client node behavior and state

Each client maintains:

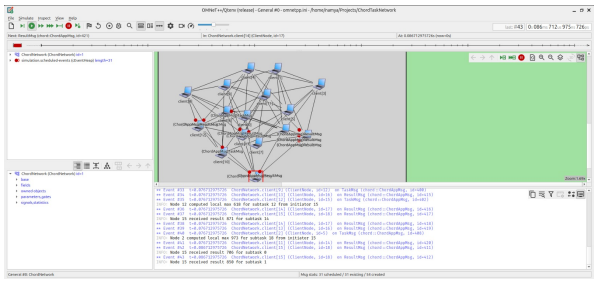
- Counters for received subtask results and the current aggregated maximum.
- A gossip message log ML implemented as a set of $(originIP, timestamp)$.
- A termination flag used to drop further messages once completion gossips from all nodes have been observed.

9 Results and Output

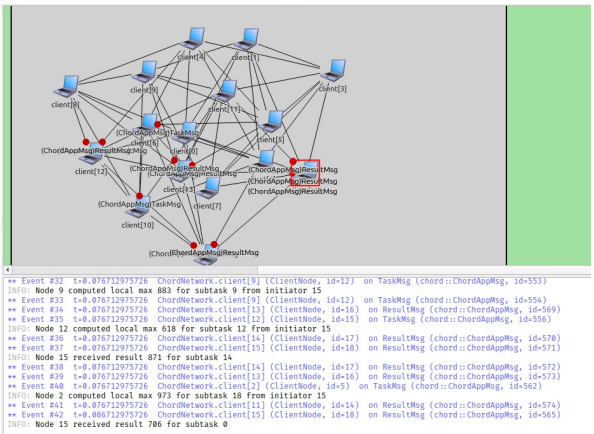
9.1 Observed output structure

The simulation logs to both the console and `outputfile.txt`. The output includes:

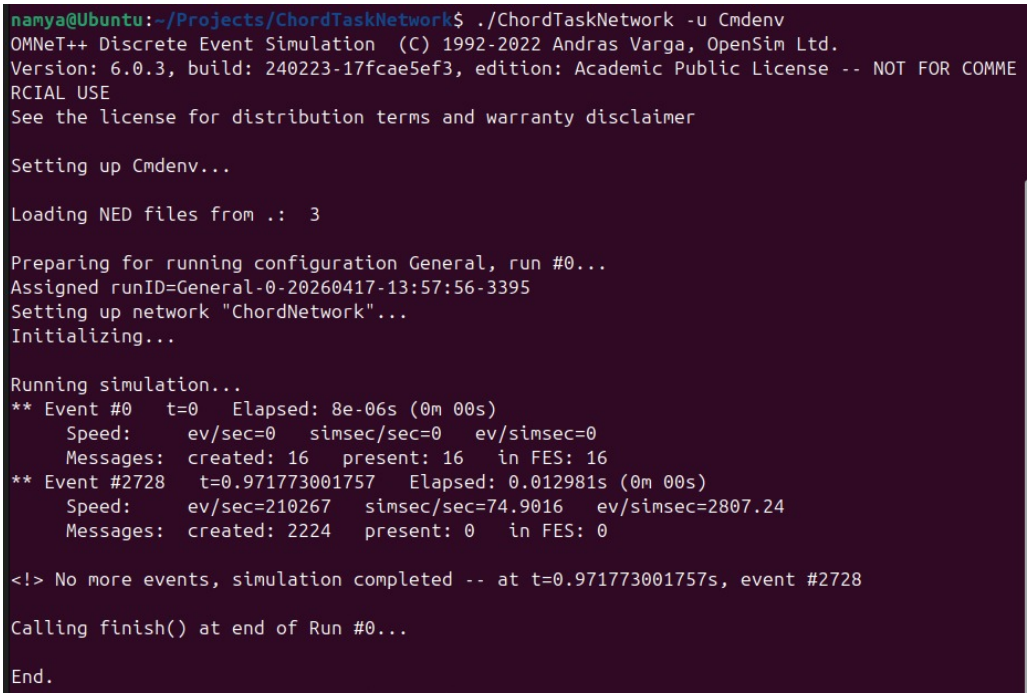
- Task initiation messages (node id, k, x).
- Per-subtask local maximum computed at the responsible node.
- Result receipts and final consolidated maximum at the initiator.
- First-time gossip receipts with both origin timestamp and local receipt timestamp.



(a) Overlay visualization (view 1).



(b) Overlay visualization (view 2).



```
namya@Ubuntu:~/Projects/ChordTaskNetwork$ ./ChordTaskNetwork -u Cmdenv
OMNeT++ Discrete Event Simulation  (C) 1992-2022 Andras Varga, OpenSim Ltd.
Version: 6.0.3, build: 240223-17fcae5ef3, edition: Academic Public License -- NOT FOR COMME
RCIAL USE
See the license for distribution terms and warranty disclaimer

Setting up Cmdenv...

Loading NED files from .:  3

Preparing for running configuration General, run #0...
Assigned runID=General-0-20260417-13:57:56-3395
Setting up network "ChordNetwork"...
Initializing...

Running simulation...
** Event #0   t=0   Elapsed: 8e-06s (0m 00s)
     Speed:     ev/sec=0   simsec/sec=0   ev/simsec=0
     Messages:  created: 16   present: 16   in FES: 16
** Event #2728   t=0.971773001757   Elapsed: 0.012981s (0m 00s)
     Speed:     ev/sec=210267   simsec/sec=74.9016   ev/simsec=2807.24
     Messages:  created: 2224   present: 0   in FES: 0

<!> No more events, simulation completed -- at t=0.971773001757s, event #2728

Calling finish() at end of Run #0...

End.
```

(c) Terminal/console output snapshot.

Figure 5: Screenshots from a sample run: network visualization and the corresponding console-style logs.

9.2 Example excerpt

Listing 1: Excerpt of the recorded output illustrating distributed max computation and subsequent gossip dissemination.

```
Node 15 initiating task, splitting 100 elements into 20 parts.
Node 13 computed local max 1000 for subtask 13 from initiator 15
...
Task Complete on Node 15. Conclusive Max: 1000
Node 0 recvd GOSSIP message <0.106713>:<15>:<15>, Local Timestamp: 0.116713, Received
    From: 15
...
```

9.3 Interpretation

The excerpt illustrates the full lifecycle: subtasks are executed at their assigned destinations, results are routed back to the initiator, and a completion gossip spreads through the overlay. Differences between the origin timestamp and each node's local receipt timestamp are expected due to hop-by-hop delay channels and discrete-event scheduling.

10 Conclusion

Overall, the simulation meets the core goals of the assignment: it distributes a task across peers, routes subtasks and results over a ring augmented with CHORD-like shortcuts, and disseminates completion using a loop-free gossip protocol. The use of finger links combined with greedy distance-based forwarding provides the intended $O(\log N)$ routing behavior in typical cases, while the message-log mechanism prevents gossip loops and redundant forwarding.

References

[1] A. Varga and OMNeT++ Community, *OMNeT++ Simulation Manual and API Reference*. https://doc.omnetpp.org/.

[2] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, "Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications," *ACM SIGCOMM*, 2001.

[3] CSL3080 Assignment–2: Distributed Task Execution and Gossip Protocol Simulation (provided handout / task specification).

[4] Project README (build/run instructions and troubleshooting), `README.md` (included with the submission).